



Symbiosis Institute of Technology, Pune

CSE- Academic Year 2026-27

Java Programming – Lab Batch 2025-29

Lab Assignment No.:	
Name of Student	Akshit Gawade
PRN No.	25070122029
Batch	2025-2029
Class	A1
Academic Year & Semester	Second Year Semester III
Date of Submission	11/7/2026
Title of Assignment:	Write a Java program to demonstrate the use of abstract classes and abstract methods.
Problem Statement	Write a Java program to demonstrate the use of abstract classes and abstract methods.
Learning Objectives	To implement abstraction using abstract classes and abstract methods.
Theory	Abstract classes provide partial implementation and abstract methods must be implemented by subclasses. Abstraction hides implementation details while exposing essential functionality.

Program Code

EX1

```
1  abstract class Payment {    Ex1.java is a non-project file, only syntax error
2      abstract void pay(double amount);
3
4      void display() {
5          System.out.println(x: "Payment Method");
6      }
7  }
8
9  class CreditCard extends Payment {
10     void pay(double amount) {
11         System.out.println("Paid Rs." + amount + " using Credit Card.");
12     }
13 }
14
15 class UPI extends Payment {
16     void pay(double amount) {
17         System.out.println("Paid Rs." + amount + " using UPI.");
18     }
19 }
20
21 public class Ex1 {
22     Run | Debug
23     public static void main(String[] args) {
24         Payment p1 = new CreditCard();
25         Payment p2 = new UPI();
26
27         p1.display();
28         p1.pay(amount: 3000);
29
30         System.out.println();
31
32         p2.display();
33         p2.pay(amount: 500);
34     }
35 }
```

EX2

```
1  abstract class FoodOrder {    Ex2.java is a non-project file, only synt
2
3      double amount;
4
5      FoodOrder(double amount) {
6          this.amount = amount;
7      }
8
9      // Abstract method
10     abstract void calculateBill();
11 }
12
13 class DineInOrder extends FoodOrder {
14
15     DineInOrder(double amount) {
16         super(amount);
17     }
18
19     void calculateBill() {
20         double total = amount + (amount * 0.10); // 10% service charge
21         System.out.println("Dine-In Bill: Rs" + total);
22     }
23 }
24
25 class TakeAwayOrder extends FoodOrder {
26
27     TakeAwayOrder(double amount) {
28         super(amount);
29     }
30
31     void calculateBill() {
32         double total = amount + 30; // ₹30 packing charge
33         System.out.println("Take Away Bill: Rs" + total);
34     }
35 }
36
37 public class Ex2 {
38     Run | Debug
39     public static void main(String[] args) {
40
41         FoodOrder order1 = new DineInOrder(amount: 200);
42         FoodOrder order2 = new TakeAwayOrder(amount: 300);
43
44         order1.calculateBill();
45         order2.calculateBill();
46     }
47 }
```

Output
Screenshots

Ex1

```

PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 10> c:; cd 'c:\U
eDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\
Payment Method
Paid Rs.3000.0 using Credit Card.

Payment Method
Paid Rs.500.0 using UPI.
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 10>

```

Ex2

```

PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 10> c:; cd 'c:\U
eDetailsInExceptionMessages' '-cp' 'C:\Users\Nilima\AppData\Roaming\
Dine-In Bill: Rs220.0
Take Away Bill: Rs330.0
PS C:\Users\Nilima\Documents\JAVA PROJECTS\LAB EXP 10>

```

Conclusion

This assignment helped in understanding the concepts of abstract classes and abstract methods in Java. It demonstrated how abstraction hides implementation details while exposing only the essential functionality, allowing subclasses to provide their own implementations of abstract methods. The implementation improved my understanding of object-oriented programming principles such as abstraction, inheritance, and polymorphism, resulting in more flexible and maintainable code. These concepts are widely used in developing desktop applications, web applications, Android applications, enterprise software, banking systems, and other real-world Java applications that require reusable, modular, and scalable software design.